A simple QML example

Let's build a quantum algorithm for binary classification. We'll use a QSVM to classify data into two categories.

Here are the steps to be followed to build the QSVM.

Prepare the data: Convert classical data into quantum states using quantum feature maps.

Design the circuit: Build a quantum circuit to process the data.

Train the model: Use a quantum optimiser to find the best hyperparameters.

Predict outputs: Measure the qubits to predict class labels.

The pseudo code for QSVM is:

```
Initialize Quantum Environment
Import quantum libraries (e.g., Qiskit)

Define feature_map:
    Input: Classical data
    Convert data to quantum states using encoding gates

Define quantum_kernel:
    Input: Quantum states
    Apply quantum operations to compute kernel matrix

Train QSVM:
    Input: Kernel matrix and class labels
    Optimise parameters using a classical-quantum hybrid
algorithm

Predict:
    Input: New data
    Use trained QSVM to classify data
    Output: Predicted class labels
```

Here's how the pseudo code translates into Python:

```
from qiskit import Aer, QuantumCircuit
from qiskit.circuit.library import ZZFeatureMap
from qiskit_machine_learning.algorithms import QSVC
from qiskit_machine_learning.kernels import QuantumKernel
```

Step 1: Data preparation

```
feature_map = ZZFeatureMap(feature_dimension=2, reps=2,
entanglement='linear')
```

Step 2: Quantum kernel

```
quantum_kernel = QuantumKernel(feature_map=feature_map,
quantum_instance=Aer.get_backend('statevector_simulator'))
```

Step 3: Train QSVM

```
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
```

Generate synthetic data

```
X, y = make_classification(n_samples=100, n_features=2, n_
classes=2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Train QSVM

```
qsvc = QSVC(quantum_kernel=quantum_kernel)
qsvc.fit(X_train, y_train)
```

Step 4: Predictions

```
predictions = qsvc.predict(X_test)
print("Predicted classes:", predictions)
```

Advantages of QML

Speed: Quantum computing accelerates tasks like matrix inversion and optimisation.

Scalability: QML helps process high-dimensional data efficiently.

Accuracy: It enhances performance in problems like clustering and classification.

The challenges

Despite its potential, QML faces a few challenges.

Hardware limitations: Quantum computers are still in their infancy.

Error correction: Quantum systems are prone to noise and errors.

Complexity: QML requires expertise in both quantum mechanics and machine learning.

Quantum machine learning opens the door to solving problems previously thought unsolvable. While still in its early stages, QML is set to transform industries like healthcare, finance, and logistics. By understanding quantum algorithms and implementing them in AI models, developers can harness this cutting-edge technology to build faster and smarter systems. END

By: Dr Shubham Mahajan

The author is assistant professor at Amity University, Haryana. His research interests primarily lie in image processing, video compression, image segmentation, fuzzy entropy, data mining, machine learning, and robotics.